

A*算法8数码问题实验报告

目录

一、实验介绍

二、算法介绍与设计

1. A*算法

a. A*算法思想——从盲目搜索到启发式搜索

i. 盲目搜索

ii. 启发式搜索

b. A*算法实现步骤与设计细节

i. 实现步骤

ii. 设计细节

1. 节点扩展

2. 启发函数

3. 回溯路径

2. IDA*算法

a. IDA*算法思想

i. 迭代加深搜索

ii. IDA*算法——采用了迭代加深算法的 A * 算法

b. IDA*算法实现

三、界面设计

1. 开始界面

2. 游戏界面

3. 设置界面

a. 模式设置

b. 难度设置

c. 对手设置（算法设置）

d. 设置始终状态

四、实验结果分析

五、实验中遇到的问题以及解决方案

六、讨论与思考

1. 启发函数的选择
2. 算法性能评估
3. 实际应用前景
4. 未来工作展望

一、实验介绍

8数码问题是一种非常典型的组合优化问题，广泛用于评估搜索算法的性能，该问题由一个3x3的方格构成，内含编号1到8的数字及一个空格，通过移动数字将其排列成特定顺序，即达到目标状态。在本实验中，我们**利用A*算法和IDA*算法解决了经典的8数码问题，并与DFS、BFS等经典算法进行了性能比较，开发了友好的图形交互界面，并为用户提供了可以自由选定的拼图目标和拼图界面。**

本文将详细介绍A*算法和IDA*算法在8数码问题中的实现细节，包括启发函数的选择、状态空间的探索以及算法性能的评估等等，为深入理解智能搜索算法提供实验依据。

二、算法介绍与设计

1. A*算法

1.1 A*算法思想——从盲目搜索到启发式搜索

A*算法本质上是一种最短路径搜索算法。迄今为止，我们学习到的最常见的几种路径搜索算法有深度优先搜索（Depth-First Search, DFS）、广度优先搜索(Breadth-First Search, BFS) 以及Dijkstra算法。

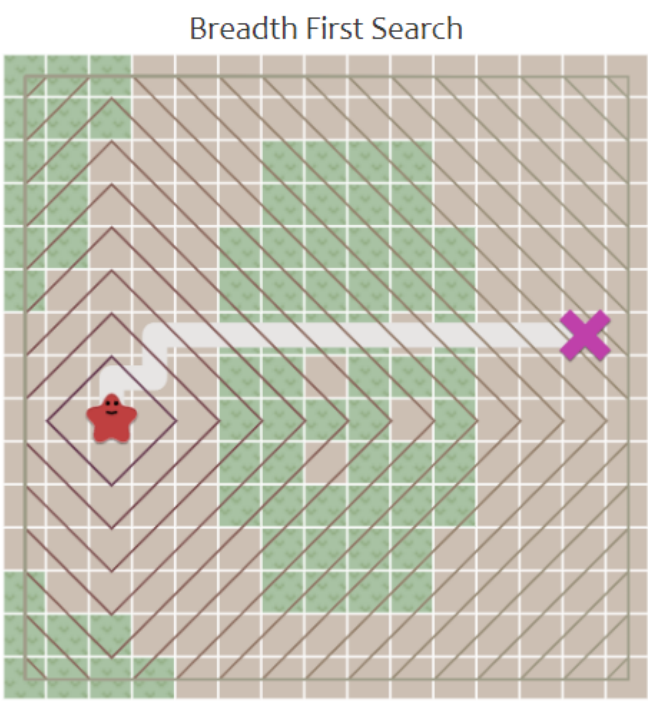
1.1.1 盲目搜索

BFS 算法思想为从起始节点开始，先访问该节点，然后依次访问其所有相邻节点，再访问这些相邻节点的相邻节点，以此类推，直到遍历完所有节点或找到目标节点。它的优点在于BFS是系统化的搜索，可以保证在无权图中找到从起点到终点的最短路径。然而，BFS的需要存储所有节点的队列，空间复杂度很高。

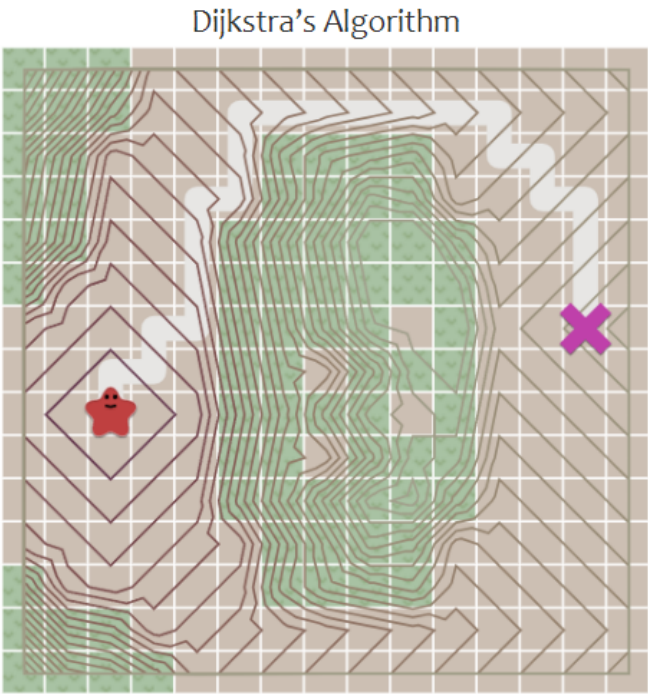
DFS 的算法思想为从根节点开始，沿着树的分支一直往下走，直到不能再继续为止，然后回溯到上一个节点并继续未完成的搜索，直到遍历完所有节点或找到目标节点。DFS 的优点在于其内存需求相对较低。然而，DFS 在无权图中不能保证找到最短路径，并且在存在环的图中，如果没有适当的访问标记，可能会陷入死循环。

Dijkstra 算法是通过逐步扩展最短路径的方式找到从起点到所有其他节点的最短路径。Dijkstra 算法的优点在于它能在非负权重图中保证找到最短路径。然而，Dijkstra 的效率依赖于实现方式，使用简

单数组实现时时间复杂度较高。



广度优先搜索



Dijkstra算法

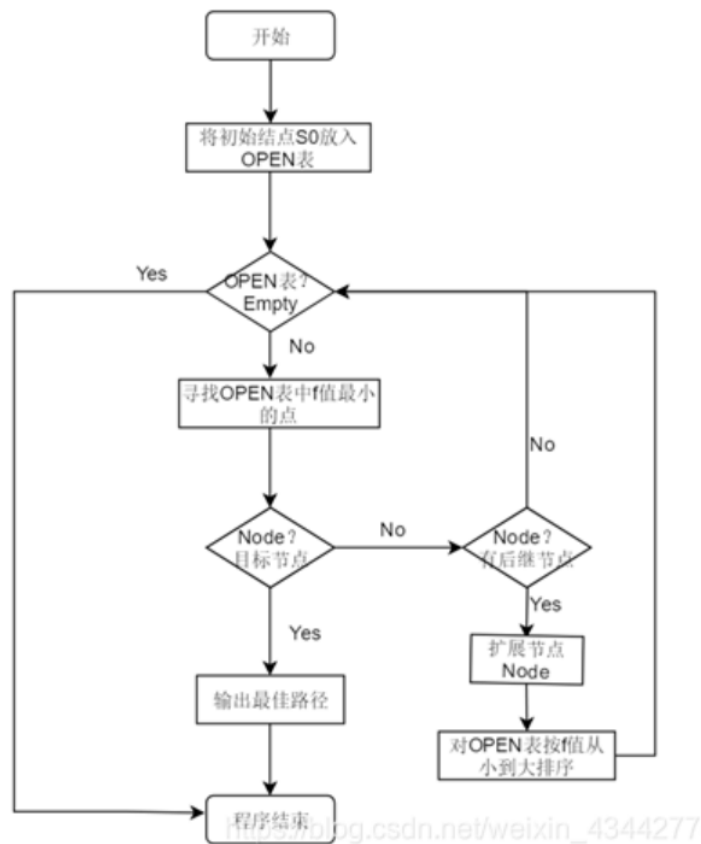
以上三种搜索算法最大的缺陷实际上是不能有效利用与问题相关的信息即盲目地、发散式地、漫无目的地去搜索，这样的搜索效率注定不会很高，而启发式搜索很好的解决了这一问题。

1.1.2 启发式搜索

启发式搜索是人工智能一种搜索技术，启发式是一个经验法则，它会导向一个解决方案。盲目搜索对前方的情况是未知的，他们不知道搜索目标是什么，没有任何关于要去目标那里的信息，除非碰巧偶然遇到了目标。我们现在定义一种关于启发式信息的启发式函数 $h(n)$ ，它用来评价哪些节点最有希望的是一个我们要找的节点，这里 n 为节点， $h(n)$ 是从节点 n 的目标节点路径的估计成本。

我们首先考察一下贪婪优先搜索算法。在Dijkstra算法中，我们使用从开始的实际距离进行优先级队列排序。而在贪婪最佳优先搜索中，我们使用到目标的估计距离进行优先级队列排序。最接近目标的位置将首先被探索，算法其余部分与Dijkstra相同。

我们将算法再次比较，如下图，可以看到A* 算法非常高效。



在具体代码实现中，我们用最小堆heap，查询字典heap_entry，集合explored共同实现上述描述的OPEN表，同时为了减少空间复杂度，我们为所有节点状态对象添加父节点和移动方向属性，同时设为全局变量，来代替掉CLOSED表的保存。具体代码实现如下：

```

1  # A *算法
2  def ast(start_state):
3      global max_frontier_size, goal_node, max_search_depth
4      # 初始化表示已探索的节点集合explored，探索过的节点的最小堆heap，记录heap每一个节点的
      堆条目heap_entry
5      explored, heap, heap_entry = set(), list(), {}
6
7      # 计算初始状态的估价函数值，创建根节点的状态
8      key = h(start_state)
9      root = State(start_state, None, None, 0, 0, key)
10
11     # heap堆的堆条目为（估价函数值，父节点到本节点的移动方式，节点状态）
12     entry = (key, 0, root)
13     heappush(heap, entry)
14     heap_entry[root.map] = entry
15
16     # 当堆不为空时开始循环
17     while heap:
18         # 拿出估价函数值最小的节点
19         node = heappop(heap)
20         explored.add(node[2].map)

```

```

21
22     # 如果该节点为目标节点，则路径搜索完成
23     if node[2].state == goal_state:
24         goal_node = node[2]
25         return heap
26
27     # 如果该节点不是目标节点，则遍历该节点的所有邻居节点
28     neighbors = expand(node[2])
29     for neighbor in neighbors:
30         # 创建邻居节点的堆条目
31         neighbor.key = neighbor.cost + h(neighbor.state)
32         entry = (neighbor.key, neighbor.move, neighbor)
33
34         # 如果该邻居节点没有被探索到，则加入最小堆，同时加入已探索过的节点的集合
35         if neighbor.map not in explored:
36             heappush(heap, entry)
37             explored.add(neighbor.map)
38             heap_entry[neighbor.map] = entry
39
40             if neighbor.depth > max_search_depth:
41                 max_search_depth += 1
42
43         # 如果该邻居节点已经被探索过，并且当前路径该节点的估价函数值小于之前路径该节
点的估价函数
44         elif neighbor.map in heap_entry and neighbor.key <
heap_entry[neighbor.map][2].key:
45             # 拿到该节点的索引
46             hindex = heap.index((heap_entry[neighbor.map][2].key,
47                                 heap_entry[neighbor.map][2].move,
48                                 heap_entry[neighbor.map][2]))
49             # 重新设置该节点估价函数以及父节点
50             heap[int(hindex)] = entry
51             heap_entry[neighbor.map] = entry
52             heapify(heap)
53
54         if len(heap) > max_frontier_size:
55             max_frontier_size = len(heap)

```

1.2.2 设计细节

这一部分将介绍我们实现A*算法的设计细节包括节点状态的设置、节点的扩展以及启发函数的设置。

1.2.2.1 状态设置

对于每个节点状态，它的属性为：

- a. **state**: 在3*3方格中, 当前状态从左上到右下每一个拼图块是什么数字, 例如[0, 8, 7, 6, 5, 4, 3, 2, 1]。
- b. **parent**: 表示父节点的状态对象, 类型为State。
- c. **move**: 表示父节点移动空格至当前节点状态时, 空格的移动方向, "1" "2" "3" "4" 分别对应空格向上、下、左、右移动。
- d. **depth, cost**: 表示从起始节点到目前节点耗费的步数。
- e. **key**: 当前节点的估价函数值, 即f(n)。

```
1 class State:
2     def __init__(self, state, parent, move, depth, cost, key):
3         self.state = state
4         self.parent = parent
5         self.move = move
6         self.depth = depth
7         self.cost = cost
8         self.key = key
9         if self.state:
10             self.map = ''.join(str(e) for e in self.state)
11
12     def __eq__(self, other):
13         return self.map == other.map
14
15     def __lt__(self, other):
16         return self.map < other.map
```

1.2.2.2 节点扩展

当我们从OPEN表中拿出估价函数值最小的节点后, 要遍历它的所有邻居。这个过程实际上是创建邻居节点对象的过程, 分为两步。第一步是构建上下左右所有移动可能后节点的状态编码, 同时注意空格在顶行、底行、最左侧一列以及最右侧一列的特殊情况; 第二步是构建邻居节点, 邻居节点的父节点设为当前节点, 实际代价为父节点的实际代价+1。

```
1 # 创建当前节点的所有邻居节点状态对象
2 def expand(node):
3     neighbors = list()
4     # 遍历当前节点上下左右所有邻居
5     neighbors.append(State(move(node.state, 1), node, 1, node.depth + 1,
6                             node.cost + 1, 0))
7     neighbors.append(State(move(node.state, 2), node, 2, node.depth + 1,
8                             node.cost + 1, 0))
9     neighbors.append(State(move(node.state, 3), node, 3, node.depth + 1,
10                             node.cost + 1, 0))
```



```

8     neighbors.append(State(move(node.state, 4), node, 4, node.depth + 1,
node.cost + 1, 0))
9
10    nodes = [neighbor for neighbor in neighbors if neighbor.state]
11    return nodes
12
13    # 拼图移动函数的输入为（移动前的状态码，移动的方向），返回值为移动后的状态码
14    def move(state, position):
15        # 复制当前状态码，避免修改原始状态码
16        new_state = state[:]
17        # 找到目前状态下空格 (0) 所在的位置
18        index = new_state.index(0)
19
20        # 1 代表空格向上移动
21        if position == 1:
22            # 检查空格是否在地图顶行
23            if index not in range(0, board_side):
24                # 将空格与上方元素交换
25                temp = new_state[index - board_side]
26                new_state[index - board_side] = new_state[index]
27                new_state[index] = temp
28                return new_state
29            else:
30                return None
31
32        # 2 代表空格向下移动
33        if position == 2:
34            # 检查空格是否在地图底行
35            if index not in range(board_len - board_side, board_len):
36                # 将空格与下方元素交换
37                temp = new_state[index + board_side]
38                new_state[index + board_side] = new_state[index]
39                new_state[index] = temp
40                return new_state
41            else:
42                return None
43
44        # 3 代表空格向左移动
45        if position == 3:
46            # 检查空格是否在地图最左侧一列
47            if index not in range(0, board_len, board_side):
48                # 将空格与左侧元素交换
49                temp = new_state[index - 1]
50                new_state[index - 1] = new_state[index]
51                new_state[index] = temp
52                return new_state
53            else:

```

```

54         return None
55
56     # 4 代表空格向右移动
57     if position == 4:
58         # 检查空格是否在地图最右侧一列
59         if index not in range(board_side - 1, board_len, board_side):
60             # 将空格与右侧元素交换
61             temp = new_state[index + 1]
62             new_state[index + 1] = new_state[index]
63             new_state[index] = temp
64             return new_state
65     else:
66         return None

```

1.2.2.3 启发函数

通过前面对A*算法的介绍我们可以知道，启发函数的选取直接影响着A*算法的性能。为此，我们经过调研，采用了以下四种启发函数进行实验比较，分别有：

- a. 曼哈顿距离：在欧几里得空间的固定直角坐标系上两点所形成的线段对轴产生的投影的距离总和。简单来说，就是两点横纵坐标差的和。

```

1  # 启发函数h(n)—曼哈顿距离
2  def Manhattan_distance(state):
3      return sum(abs(b % board_side - g % board_side) + abs(b // board_side -
4                      g // board_side)
5                  for b, g in ((state.index(i), goal_state.index(i)) for i in
6                              range(1, board_len)))

```

- b. 错位数：将起始状态和目标状态相比较，对不在目标状态的拼图块个数计数。

```

1  # 启发函数h(n)—错位数
2  def misplaced_tiles(state):
3      return sum(s != g for s, g in zip(state, goal_state))

```

- c. 欧拉距离：欧几里得空间中两点之间的欧几里得距离是指连接这两点的线段的长度。简单来说就是直角坐标系中两点之间的直线距离。

```

1  # 启发函数h(n)—欧拉距离
2  def euclidean_distance(state):
3      return sum(((b % board_side - g % board_side) ** 2 + (b // board_side -
4                      g // board_side) ** 2) ** 0.5

```

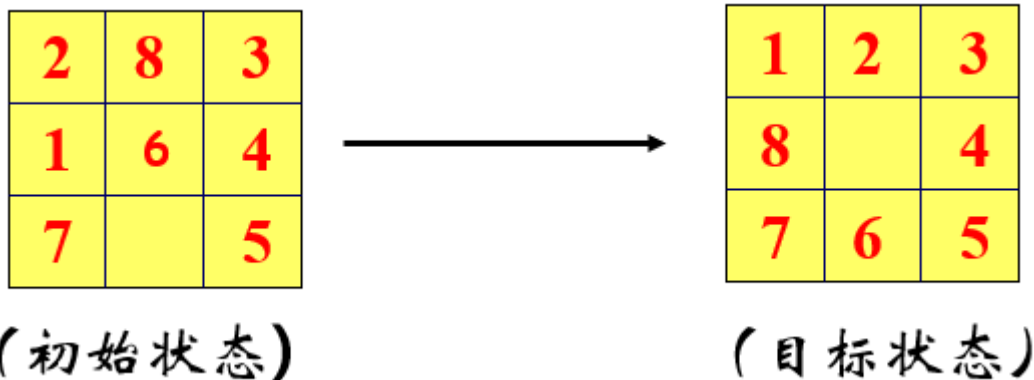
```
4         for b, g in ((state.index(i), goal_state.index(i)) for i in
    range(1, board_len)))
```

- d. 切比雪夫距离：切比雪夫距离定义为其各坐标数值差的最大值。以 $(x(1), y(1))$ 和 $(x(2), y(2))$ 二点为例，其切比雪夫距离为 $\max(|x(2)-x(1)|, |y(2)-y(1)|)$ 。

```
1 # 启发函数h(n)—切比雪夫距离
2 def chebyshev_distance(state):
3     return max(abs(b % board_side - g % board_side) + abs(b // board_side -
    g // board_side)
4         for b, g in ((state.index(i), goal_state.index(i)) for i in
    range(1, board_len)))
```

同时，我们也考察到了采用 $h(n) = P(n) + 3S(n)$ 作为启发函数，其中：

- a. $P(n)$ 为某个数字距离目标的绝对距离；
- b. $S(n)$ 为顺序分值：
 - i. 2, 非中间位置，而且没有被顺序跟随
 - ii. 1, 中间位置；
 - iii. 0, 其它；



但是在很多情况下，拼图的目标状态并不是1~8从空格左边开始顺时针旋转（如上图），所以考虑到本项目的泛化性能即不单单解决8数码问题而是解决真正的拼图问题，我们不对该种启发函数进行研究。

1.2.2.4 回溯路径

找到目标节点（完成拼图）后，我们要从目标节点回溯到起始节点，以获取拼图的移动路径（空格的移动路径）。通过检查每个节点的移动方向，逐步回溯到起始节点，同时记录每一步的移动方向，最终得到完整的移动路径。

```

1 # 从目标节点回溯得到拼图步骤（空格的移动路径）
2 def backtrace():
3     # 从目标节点开始回溯
4     current_node = goal_node
5
6     # 检查是否回溯至起始节点
7     while initial_state != current_node.state:
8         # "1" "2" "3" "4" 分别对应空格向上、下、左、右移动
9         if current_node.move == 1:
10             movement = 'Up'
11         elif current_node.move == 2:
12             movement = 'Down'
13         elif current_node.move == 3:
14             movement = 'Left'
15         else:
16             movement = 'Right'
17
18         # 将移动方向插入在空格移动路径的第一个位置，这样可以保证最终输出的路径是从起始节
        点开始的
19         moves.insert(0, movement)
20         current_node = current_node.parent
21
22     return moves

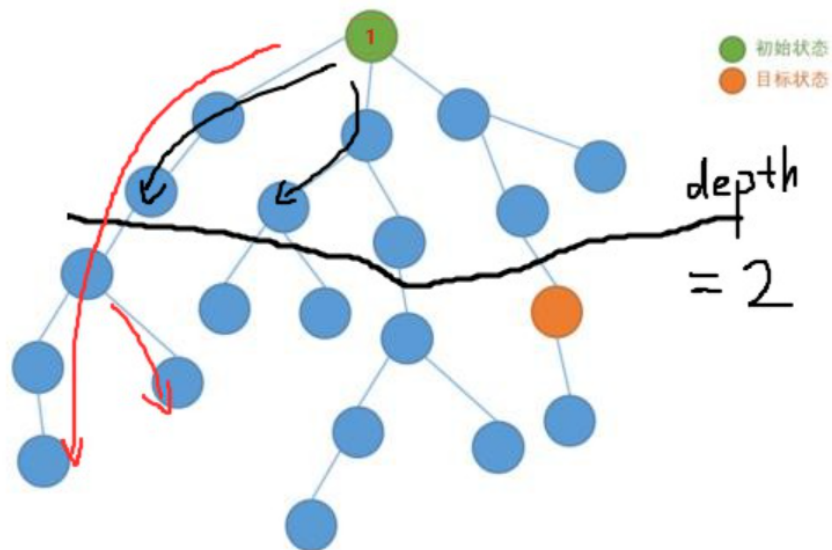
```

2. IDA*算法

2.1 IDA*算法思想

2.1.1 迭代加深搜索

在介绍IDA*算法之前，我们先介绍一下迭代加深搜索算法。迭代加深算法是在DFS搜索算法的基础上逐步加深搜索的深度，减少了了深度优先搜索的盲目性。它主要是在递归搜索函数的开头判断当前搜索的深度是否大于预定义的最大搜索深度，如果大于，就退出这一层的搜索，如果不大于，就继续进行搜索。这样最终获得的解必然是最优解。



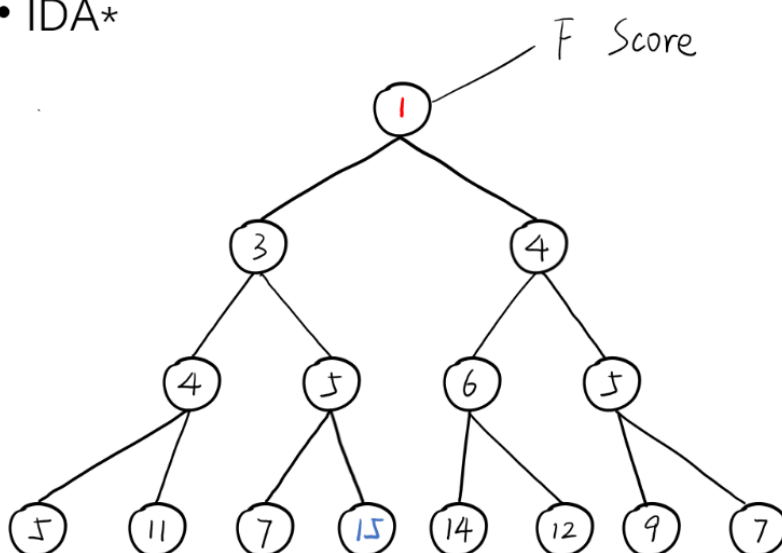
那既然是为了找最优解，为什么不用 BFS 呢？我们知道 BFS 的基础是一个队列，队列的空间复杂度很大，当状态比较多或者单个状态比较大时，使用队列的 BFS 就显出了劣势。事实上，迭代加深就类似于用 DFS 方式实现的 BFS，它的空间复杂度相对较小。

当搜索树的分支比较多时，每增加一层的搜索复杂度会出现指数级爆炸式增长，这时前面重复进行的部分所带来的复杂度几乎可以忽略，这也就是为什么迭代加深是可以近似看成 BFS 的。

2.1.2 IDA*算法——采用了迭代加深算法的 A* 算法

IDA* 的具体名称为 Iterative Deepening A*，简单来说就是采用了迭代加深算法的 A* 算法。它综合了 A* 算法的人工智能性和回溯法对空间的消耗较少的优点，该算法的最初目的是为了利用深度搜索的优势解决广度 A* 的空间问题，其代价是会产生重复搜索。

• IDA*



- ① 设置深度: 3
- ② $1 \rightarrow 3 \rightarrow 4$
- ③ $1 \rightarrow 3 \rightarrow 5$
- ④ $1 \rightarrow 4$
- ⑤ 扩展深度: 4
- ⑥ $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$
- ⑦ $1 \rightarrow 3 \rightarrow 4 \rightarrow 11$
- ⑧ $1 \rightarrow 3 \rightarrow 5$
- ⑨ $1 \rightarrow 4 \rightarrow 6$
- ⑩ $1 \rightarrow 4 \rightarrow 5$
- ⑪ 扩展深度: 5
- ⑫ ...

IDA*的基本思路是：首先将初始状态结点的H值设为阈值maxH，然后进行深度优先搜索，搜索过程中忽略所有H值大于maxH的结点；如果没有找到解，则加大阈值maxH，再重复上述搜索，直到找到一个解。在保证H值的计算满足A*算法的要求下，可以证明找到的这个解一定是最优解。在程序实现上，IDA* 要比 A* 方便，因为不需要保存结点，不需要判重复，也不需要根据 H 值对结点排序，占用空间小。

在IDA*算法中也使用合适的估价函数，来评估与目标状态的距离。在一般的问题中IDA*算法的使用为：

当前局面的启发函数值+当前的搜索深度 > 预定义的最大搜索深度时，就进行剪枝。

2.2 IDA*算法实现

1. `ida(start_state)`：IDA*算法的入口函数，使用迭代加深的方式来搜索最优路径。首先，设置初始阈值为启发函数值，并进入一个无限循环。在每轮迭代中，调用 `dls_mod(start_state, threshold)` 函数进行深度限制搜索，直到找到解或达到阈值。如果找到解，则返回解路径；否则，更新阈值为下一轮迭代的最小成本。

```
1 #IDA*算法
2 def ida(start_state):
3     global costs
4     # 初始阈值为启发函数值
5     threshold = h(start_state)
6
7     while True:
8         # 在深度限制搜索中寻找路径
9         response = dls_mod(start_state, threshold)
10        if type(response) is list:
11            return response
12
13        # 更新阈值为下一轮迭代的最小成本
14        threshold = response
15        # 清空成本集合
16        costs = set()
```

2. `dls_mod(start_state, threshold)`：实现深度限制搜索的逻辑。它使用一个栈来维护搜索状态，并使用一个集合来记录已探索的节点。在每次迭代中，从栈中弹出一个节点，并将其加入到已探索集合中。如果当前节点是目标节点，则返回路径。否则，如果当前节点的成本超过阈值，则记录当前节点的成本；如果当前节点深度小于阈值，则扩展当前节点的邻居节点，并将其加入到栈中。在扩展邻居节点时，同样需要判断节点是否已经探索过，以避免重复搜索相同状态。最终，如果栈为空，则返回下一轮迭代的最小成本。

```

2 def dls_mod(start_state, threshold):
3     global max_frontier_size, goal_node, max_search_depth, costs
4     # 初始化已探索集合和搜索栈
5     explored, stack = set(), [State(start_state, None, None, 0, 0, threshold)]
6
7     while stack:
8         # 弹出栈顶节点
9         node = stack.pop()
10        # 添加到已探索集合
11        explored.add(node.map)
12
13        # 如果当前节点为目标节点，则返回路径
14        if node.state == goal_state:
15            goal_node = node
16            return stack
17
18        # 如果节点的成本大于当前阈值，则记录成本
19        if node.key > threshold:
20            costs.add(node.key)
21
22        # 如果节点深度小于阈值，则继续扩展
23        if node.depth < threshold:
24            neighbors = reversed(expand(node))
25            for neighbor in neighbors:
26                if neighbor.map not in explored:
27                    neighbor.key = neighbor.cost + h(neighbor.state)
28                    stack.append(neighbor)
29                    explored.add(neighbor.map)
30
31            # 更新最大搜索深度
32            if neighbor.depth > max_search_depth:
33                max_search_depth += 1
34
35            if len(stack) > max_frontier_size:
36                max_frontier_size = len(stack)
37
38        # 返回下一轮迭代的最小成本
39        return min(costs)

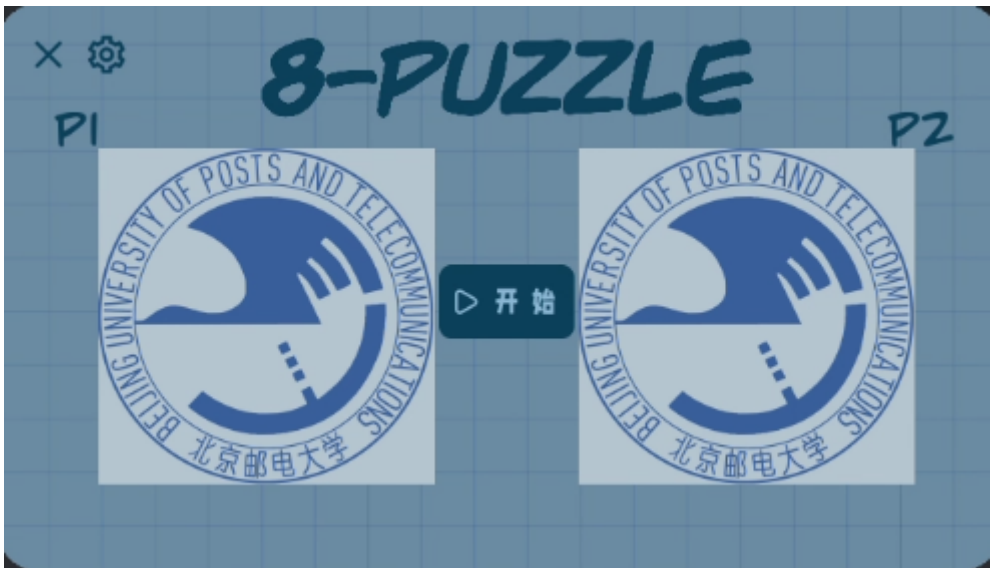
```

三、界面设计

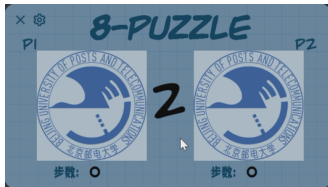
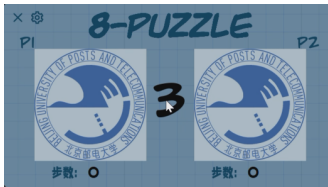
1. 游戏流程

本部分将对我们设计的8数码问题游戏界面做一个简单介绍，主要分为开始界面、游戏界面以及设置界面。更详细的部分请观看演示视频。

1.1 开始界面

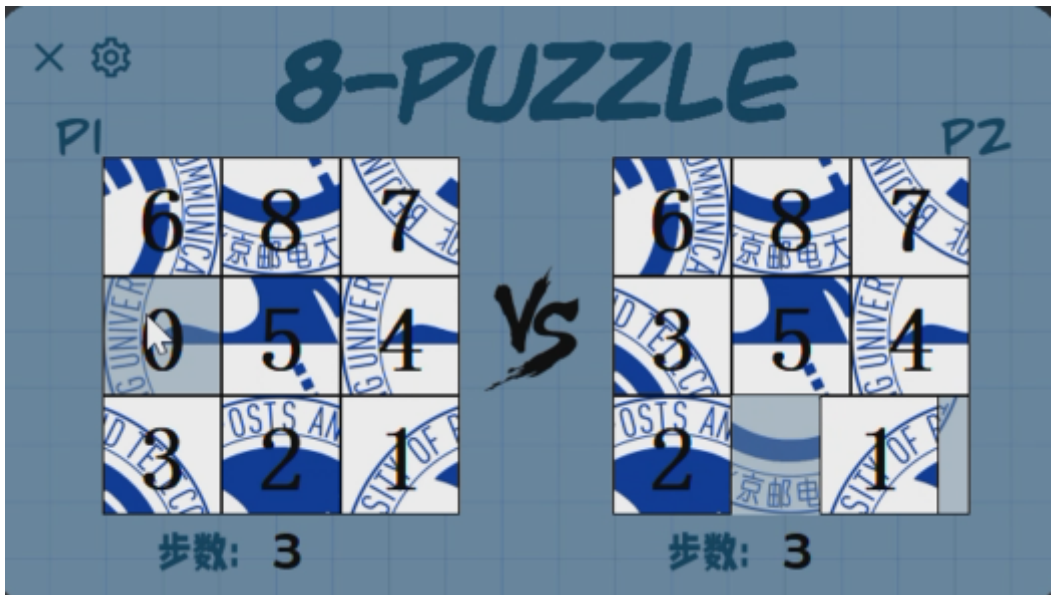


开始界面由两张拼图，标题“8-PUZZLE”和“开始”键组成。点击“开始”，显示倒计时界面。



1.2 游戏界面

游戏界面的左边为人类玩家，右边为电脑玩家或人类玩家。每一个玩家下面都有步数显示，每移动一次拼图块步数加1。



当双方玩家有一方结束时，界面将弹出输赢窗口，显示输赢情况和步数累计情况。



1.3 设置界面



1.3.1 模式设置

我们在游戏中设置了两种对战模式，一种是PVP模式，一种是PVE模式。

PVP（Player versus Player），即玩家对战模式。这种模式下，用户对两边拼图都可以操控，从而实现玩家对战。

PVE（Player versus Environment），即人机对战模式。在这种模式下，用户只被允许操纵左边的拼图，与右边的电脑进行对战。

1.3.2 难度设置

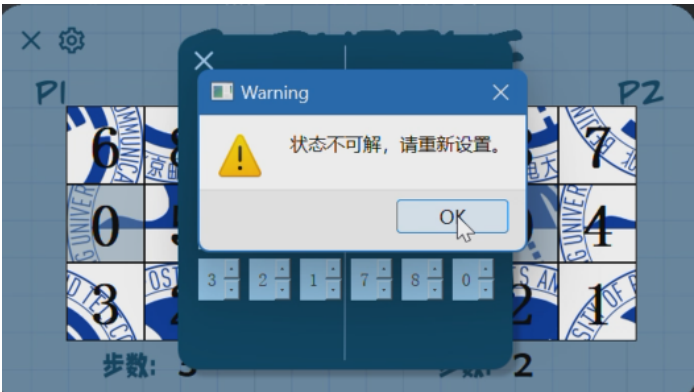
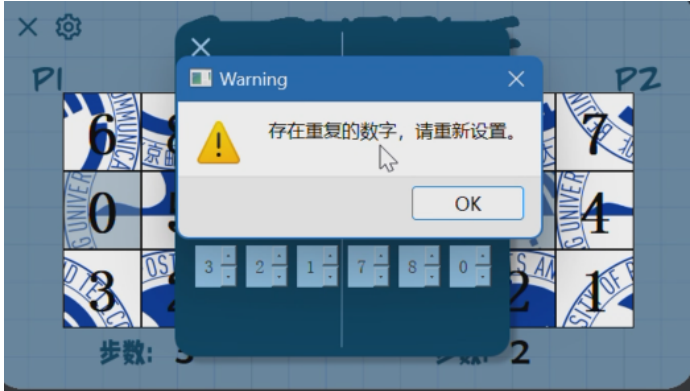
我们在游戏中设置了困难、普通、悠闲三种难度设置。难度区分体现在电脑移动拼图的速度，难度越高，拼图移动速度越快。

1.3.3 对手设置（算法设置）

对手设置部分即算法选择设置，玩家可以选择电脑使用DFS、BFS、A*、IDA*算法进行对战，不同的对手（不同的算法）完成拼图的速度也不同。

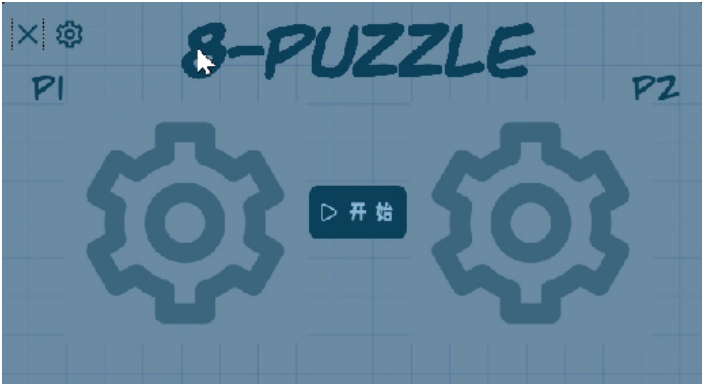
1.3.4 设置始终状态

我们在游戏中还开放了对始终状态的设置，通过更改九宫格中每一个方格的数字来设置始终状态。下图展示的是两种提示情况，一种是重复数字，一种是拼图无解，对应算法无解。



1.3.5 设置背景图片

我们还在游戏中添加了背景图片设置选项，玩家可以根据自己喜好选择图片来进行拼图游戏。



2. 软件设计

该界面设计采用典型的软件设计方法，特别是在基于图形用户界面的应用程序中使用。以下是软件设计的分析：

2.1 软件架构概览

分层设计：

界面层（UI Layer）：借助PyQt5框架，精心构建用户交互界面，涵盖窗口、控件布局与外观设计。

逻辑层 (Logic Layer) :封装游戏逻辑和功能实现, 如PuzzleBoard类, 确保游戏机制顺畅运行。

数据层 (Data Layer) :管理游戏的配置、状态和数据存储。

2.2 主要组件

主窗口类 (Ui_MainWindow) :

负责初始化和设置主窗口的布局和小部件。作为用户交互的中心点, 负责界面布局初始化与用户操作响应, 如按钮点击、游戏开始等。

自定义小部件 (PuzzleBoard) :

继承自QtWidgets.QWidget, 自定义了游戏棋盘的显示和交互功能, 实现游戏核心逻辑, 包括棋盘显示、交互与状态管理、游戏状态的初始化、更新和判断 (调用is_solvable函数实现)。

游戏配置和状态:

initialConfig()方法用于初始化游戏配置, 包括读取配置文件或设置默认值。

self.mode、self.p1_mode、self.p2_mode、self.config和self.game_started等变量用于存储当前游戏状态和配置。

2.3 功能模块设计原则

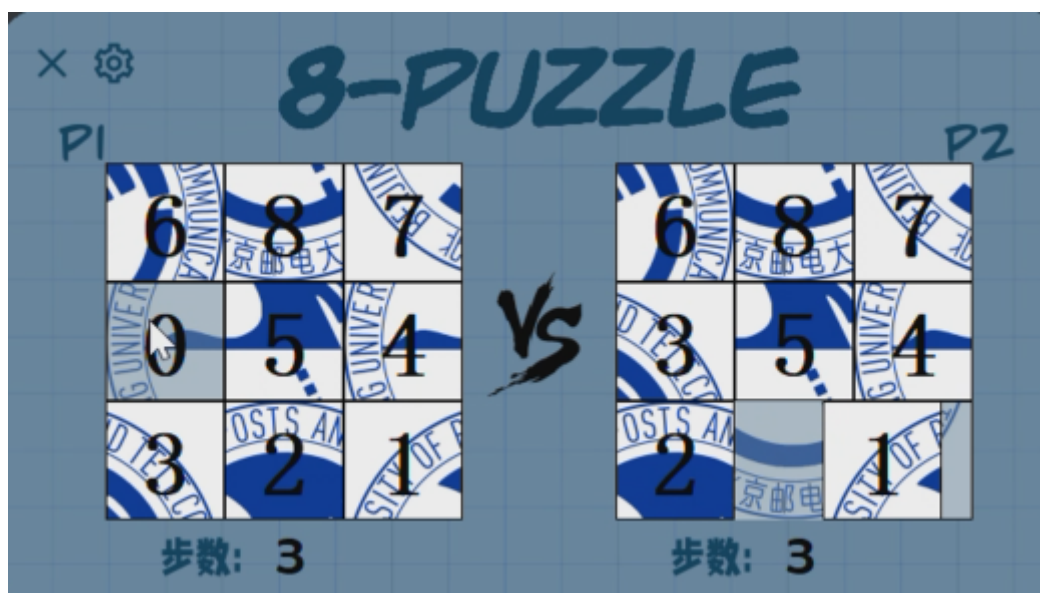
- 模块化与分层: 通过将功能划分为独立模块 (如UI模块、逻辑模块), 增强代码的维护性和扩展性。
- 面向对象实践: 运用类和对象封装, 如Ui_MainWindow和PuzzleBoard, 实现清晰的职责分配与易于管理的代码结构。
- 事件驱动编程: 采用事件与信号槽机制, 使程序能高效响应用户交互, 提升用户体验。

3. 核心组件

3.1 主窗口类 (Ui_MainWindow):

职责: 负责创建和管理主窗口及其所有子组件, 设置布局, 初始化游戏状态, 处理主要的用户交互。

图片分割函数: 定义了一个split_image(img, end_state)函数, 效果如下:



实现步骤包括如下部分：

3.1.1 打开图像并调整大小

使用PIL（Python Imaging Library）打开输入图像。获取图像的宽度和高度，计算新的图像大小，取宽度和高度中的较大值，确保图像为正方形。将图像调整为新的大小，使用Image.Resampling.LANCZOS 进行高质量的图像缩放。

```
1 image = Image.open(img) # 打开输入图像文件
2 width, height = image.size # 获取图像的宽度和高度
3 new_size = max(width, height) # 计算图像的新的尺寸（取宽度和高度的较大值）
4 image = image.resize((new_size, new_size), Image.Resampling.LANCZOS) # 将图像调整为新的尺寸
```

3.1.2 计算每个网格大小

```
1 grid_size = new_size / 3 # 计算每个网格的大小
```

3.1.3 分割图像并存储图像块:

```
1 grid_images = [] # 创建一个列表，用于存储分割后的图像块
2 for i in range(3): # 遍历行
3     for j in range(3): # 遍历列
4         left = round(j * grid_size) # 计算左边界
5         top = round(i * grid_size) # 计算上边界
6         right = round(left + grid_size) # 计算右边界
7         bottom = round(top + grid_size) # 计算下边界
8         grid_image = image.crop((left, top, right, bottom)) # 裁剪图像块
```

3.1.4 替换一个网格块为透明图像：

找到 end_state 中值为 0 的位置。使用 Image.new 创建一个透明的图像块，大小与网格块相同，并替换对应位置的图像块。

```
1 grid_images[end_state.index(0)] = Image.new('RGBA', (int(grid_size),
int(grid_size)), (0, 0, 0, 0)) # 将某个图像块替换为透明图像
```

3.1.5 保存每个图像块：

遍历 grid_images 列表，将每个图像块保存到指定路径，其中文件名由 end_state 的值确定

```
1 for i, img in enumerate(grid_images): # 遍历分割后的图像块
2     img.save(f'src/img/temp/{end_state[i]}.png') # 保存每个图像块到指定路径
```

3.2 自定义棋盘类 (PuzzleBoard):

职责: 负责显示和管理游戏棋盘，包括处理游戏逻辑和用户交互。八数码拼图游戏的核心部分，包括图片的分割、拼图板的初始化、随机打乱、用户交互和自动求解等功能。

3.2.1 PuzzleBoard 类的定义

构造函数: __init__(self, x, y, parent, size, mode, config): 初始化棋盘的位置、大小、模式和配置。

功能: 处理游戏棋盘的绘制、更新和状态管理。

- 定义信号：solutionFound（求解方案找到时发出），moveCompleted（移动完成时发出），countChanged（计数改变时发出），gameEnd（游戏结束时发出）。
- 初始化拼图板，包括图片、终态、初态、难度和模式等配置。
- 创建并初始化拼图块。

```
1 class PuzzleBoard(QWidget):
2     #以下对信号定义
3     solutionFound = pyqtSignal(list)
4     moveCompleted = pyqtSignal()
5     countChanged = pyqtSignal(int)
6     gameEnd = pyqtSignal()
7     #初始化方法
8     def __init__(self, x, y, parent=None, width=231, mode='manual',
config=None):
```

```

9         super().__init__(parent)
10        surplus = width % 3
11        if surplus != 0:
12            width = width - surplus
13        self.step_width = int(width / 3)
14        #初始化函数接收初始位置 (x, y)、父组件 parent、棋盘宽度 width、模式 mode 和配置
        config。调整 width 使其能被3整除, 并计算每个拼图块的大小 step_width。
15
16        #以下是对属性初始化, 初始化各种属性, 包括图像、状态、动画标志、难度、模式和动画对象。调用
        update_config 方法更新配置。设置初始计数和状态。设置样式表, 将背景图像设置为 bupt.jpg。
        设置组件的位置和大小。初始化 pieces 字典, 用于存储拼图块组件
17        self.img = None
18        self.end_state = None
19        self.start_state = None
20        self.isAnimating = False
21        self.difficulty = None
22        self.mode = mode
23        self.animation1 = None
24        self.config = config
25        self.update_config()
26        self._count = None
27        self.count = 0
28        self.state = list() # 8数码的状态
29        self.setStyleSheet("QWidget#Form{\n"
30                            "    image: url(:/img/bupt.jpg);\n"
31                            "}")
32        self.x = x
33        self.y = y
34        self.setGeometry(x, y, width, width)
35        self.pieces = {}
36        #创建拼图块, 使用嵌套循环创建 3x3 的拼图块, 并设置每个块的位置和大小。将每个拼图块存储在
        pieces 字典中, 键为块的索引。
37        index = 0
38        for row in range(1, 4):
39            for col in range(1, 4):
40                piece = PuzzlePiece(index, parent)
41                pos_x = x + (col - 1) * self.step_width
42                pos_y = y + (row - 1) * self.step_width
43                piece.setGeometry(QRect(pos_x, pos_y, self.step_width,
self.step_width))
44                piece.setObjectName(str(index))
45                piece.setVisible(False)
46                self.pieces[index] = piece
47                index = index + 1
48        #信号和槽连接, 将 moveCompleted 信号连接到 nextMove 槽。初始化 moveQueue, 用于存储移动
        步骤。为每个拼图块的点击事件连接到 move_by_push 方法。将 solutionFound 信号连接到
        solvePuzzle 方法。

```



```

49     self.moveCompleted.connect(self.nextMove)    # 将信号连接到nextMove槽
50     self.moveQueue = []    # 移动队列初始化
51     self.pieces[0].clicked.connect(lambda: (self.move_by_push(0)))
52     self.pieces[1].clicked.connect(lambda: (self.move_by_push(1)))
53     self.pieces[2].clicked.connect(lambda: (self.move_by_push(2)))
54     self.pieces[3].clicked.connect(lambda: (self.move_by_push(3)))
55     self.pieces[4].clicked.connect(lambda: (self.move_by_push(4)))
56     self.pieces[5].clicked.connect(lambda: (self.move_by_push(5)))
57     self.pieces[6].clicked.connect(lambda: (self.move_by_push(6)))
58     self.pieces[7].clicked.connect(lambda: (self.move_by_push(7)))
59     self.pieces[8].clicked.connect(lambda: (self.move_by_push(8)))
60     self.solutionFound.connect(self.solvePuzzle)

```

3.2.2 启动和结束游戏

- start 方法：初始化和开始游戏，准备拼图块，并根据配置决定是否自动解决拼图。
- end 方法：用于刷新界面（没有实际功能，只是更新界面）。
- restart 方法：重新开始游戏，类似于 start，但在调用前暂停任何进行中的动画。
- pause 方法：暂停当前的动画并重置动画状态。

3.2.3 配置更新

```

1     def update_config(self):
2         if self.mode == 'auto':
3             self.difficulty = self.config['difficulty']
4         else:
5             self.difficulty = 'manual'
6
7         if self.config is None:
8             self.start_state = [0, 8, 7, 6, 5, 4, 3, 2, 1]
9             self.end_state = [0, 1, 2, 3, 4, 5, 6, 7, 8]
10            self.img = 'src/img/bupt.jpg'    # 背景图片
11        else:
12            self.start_state = list(self.config['start_state'])
13            self.end_state = list(self.config['end_state'])
14            self.img = self.config['background_img']

```

- 根据配置更新初始状态、终态和难度。我们在游戏中设置了困难、普通、悠闲三种难度设置。难度区分体现在电脑移动拼图的速度，难度越高，拼图移动速度越快。

3.2.4 手动移动拼图块

- move_by_push 方法：处理手动点击移动拼图块。

- `move_by_direction` 方法：处理按方向键移动拼图块。

```
1 def move_by_push(self, index):
2     if self.mode == 'manual':
3         self._move(index)
```

`move_by_push` 方法用于处理手动点击移动拼图块。`index` 是用户点击的拼图块的索引。检查当前模式是否为手动模式 (`manual`)。如果是手动模式，则调用内部的 `_move` 方法，并将用户点击的拼图块索引传递给它，以尝试移动该拼图块。

```
1 def move_by_direction(self, direction):
2     zero_index = self.state.index(0)
3     row, col = zero_index // 3, zero_index % 3
4
5     if direction == 'Up':
6         row -= 1
7     elif direction == 'Down':
8         row += 1
9     elif direction == 'Left':
10        col -= 1
11    elif direction == 'Right':
12        col += 1
13
14    if 0 <= row <= 2 and 0 <= col <= 2:
15        index = row * 3 + col
16        self._move(self.state[index])
17    else:
18        return None
```

`move_by_direction` 方法用于处理按方向键移动拼图块。`direction` 是一个字符串，表示移动的方向 ('Up', 'Down', 'Left', 'Right')。找到空白块（值为0）的索引，并计算其在 3x3 网格中的行 (`row`) 和列 (`col`)。根据传入的方向参数，调整空白块的位置。检查调整后的行和列是否在有效范围内（0 到 2 之间）。根据传入的方向参数，调整空白块的位置。

3.2.5 求解拼图

- `solvePuzzle` 方法：开始求解拼图，并将移动队列传递给 `nextMove` 方法逐步执行。
- `nextMove` 方法：从移动队列中取出下一个移动方向并执行。

```
1 def solvePuzzle(self, moveQueue=None):
2     if not moveQueue:
```



```
3         return
4         self.moveQueue = moveQueue
5         self.nextMove()
```

此方法用于初始化解谜过程，将解谜步骤加载到移动队列中，并开始执行第一个移动。moveQueue 是一个列表，包含解谜所需的移动方向序列。检查 moveQueue 是否为空。如果为空，直接返回，不进行任何操作。将 moveQueue 保存到实例变量 self.moveQueue 中。调用 nextMove 方法，开始执行第一个移动。

```
1 def nextMove(self):
2     if self.moveQueue:
3         direction = self.moveQueue.pop(0)
4         self.move_by_direction(direction)
```

用于执行移动队列中的下一个移动指令。检查 self.moveQueue 是否为空。如果不为空，从队列中取出第一个方向指令。使用 move_by_direction 方法根据取出的方向指令移动拼图块。

3.2.6 初始化和打乱拼图块

- split_image 方法分割图片并初始化每个拼图块的图片。
- _shuffle_pieces 方法将拼图块打乱到初始状态。

```
1 def _init_pieces(self):
2     split_image(self.img, self.end_state)
3     for index in range(0, 9):
4         button = self.pieces[index]
5         button.p_img = f'src/img/temp/{index}.png'
6         button.update()
```

_init_pieces 将整张图片分割成九块并为每个拼图块设置图像。调用 split_image 函数，将图片 self.img 分割成九块，并按照 self.end_state 的顺序保存这些图像。遍历 0 到 8 的索引：获取对应的拼图块 button。设置拼图块的图像路径。

```
1 def _shuffle_pieces(self, state):
2     for pos_index, button_index in enumerate(state):
3         if pos_index == 8:
4             x, y = self._get_position(0)
5         else:
6             x, y = self._get_position(pos_index + 1)
7         self.pieces[button_index].move(x, y)
```

_shuffle_pieces 根据给定的状态 state 重新排列拼图块的位置。遍历状态列表 state。获取每个拼图块的位置坐标。调用 self.pieces[button_index].move(x, y) 移动拼图块到新的位置。更新当前拼图块状态 self.state。

3.2.7 辅助方法

- 1 _get_position 方法根据索引返回拼图块的位置。
- 2 _swap_pieces 方法执行拼图块交换动画。
- 3 _animation_finished 方法在动画完成后更新状态并发出信号。
- 4 _move 方法验证并移动拼图块。
- 5 _can_move 方法检查拼图块是否可以移动。
- 6 _check_win 方法检查是否完成拼图。

3.2.8 判断可解性

检查八数码板的初始状态和目标状态是否可解。在八数码问题中，初始状态和目标状态的可解性取决于它们的逆序对数量的奇偶性是否相同。

calculate_inversions(state) 函数：

接受一个状态 state（列表形式），计算非零元素之间的逆序对数量。

逆序对定义：如果对于两个元素 state[i] 和 state[j]，当 $i < j$ 且 $state[i] > state[j]$ 时，这两个元素构成一个逆序对。

返回逆序对的数量。

is_solvable(start_state, goal_state) 函数：

接受初始状态 start_state 和目标状态 goal_state。

使用 calculate_inversions 函数分别计算初始状态和目标状态的逆序对数量，判断初始状态的逆序对数量和目标状态的逆序对数量的奇偶性是否相同。如果相同，则返回 True，表示问题可解；否则返回 False，表示问题不可解。

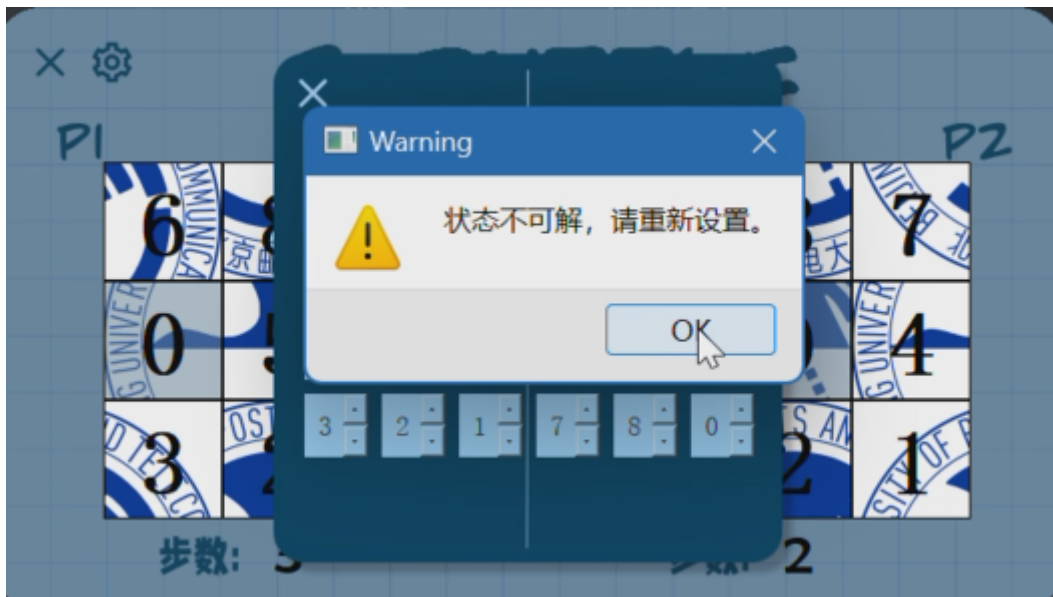
主程序部分 (__main__)：

调用 is_solvable 函数来检查一个具体的例子：

start_state = [0, 1, 2, 3, 4, 5, 6, 7, 8]，表示一个有序的初始状态（其中 0 表示空格）。

goal_state = [1, 2, 3, 4, 5, 6, 7, 8, 0]，表示一个有序的目标状态。

打印检查结果，即初始状态是否可解到达目标状态。如果不可解，将在界面中提示：



4. 软件结构

软件结构采用了面向对象设计和模块化设计原则，分为多个层次和模块来实现其功能。

主程序入口：在一个独立的文件中（如 main.py），负责初始化应用程序并启动主窗口。

用户界面层（UI Layer）：包含用于创建和管理图形用户界面的所有类和模块。主窗口类（Ui_MainWindow）和各种小部件（widgets）。

业务逻辑层（Logic Layer）：包含实现游戏逻辑的类和函数。自定义的棋盘类（PuzzleBoard）以及游戏状态的判断函数（如 is_solvable）。

数据层（Data Layer）：管理应用程序的配置和状态，包括从配置文件读取数据或保存游戏进度。

四、实验中遇到的问题以及解决方案

本次实验中遇到的最大的一个问题是8数码拼图是否有界的问题。起初我们最简单直接的想法是，如果A*算法执行过程中出现OPEN表为空，但是没有找到最终路径时，问题无解。

然而，通过数学计算可以得到8数码问题的理论最多状态节点数。假设空格可以在任意位置移动，那么对于每个位置，可以移动到上、下、左、右四个方向中的三个方向（边界位置除外）。因此，对于每个状态，可以有3个移动方向，空格有8个可能的位置，而每个位置上可以放置8个数字中的一个。所以，总的状态节点数为：

$$8! \times 9 = 181,440$$

如果我们每次判断问题是否有解，都要先完整的执行整个算法才能得出结论，效率太过低下，一定有不执行算法就可以判断是否有解的方法。

经过调研我们发现了一个定理：一个有界的状态随意交换两个数码，一定会变成无解的状态。无解的状态，随意交换两个数码，也就一定会变成有解的状态。

那么要怎么根据这个定理去判断当前的状态是否不可解呢？这个问题的难点在于A*算法没有给定一个范围，告诉你某个规模的问题在多少步内可以求解。

最后经过小组一起讨论后我们发现，要解决这个问题其实非常简单，只要判断初始状态和目标状态的逆序对数量的奇偶性即可，**如果初始状态和目标状态的逆序对数量的奇偶性相同，则问题可解；如果初始状态和目标状态的逆序对数量的奇偶性不同，则问题无解。**

具体代码实现如下：

```
1 # 计算非 0 元素（除去空格）外的逆序对
2 def calculate_inversions(state):
3     inversions = 0
4     for i in range(len(state)):
5         for j in range(i + 1, len(state)):
6             if state[i] > state[j] and state[i] != 0 and state[j] != 0:
7                 inversions += 1
8     return inversions
9
10 # 检查问题是否可解
11 def is_solvable(start_state, goal_state):
12     # 计算初始状态和目标状态的逆序对数量
13     start_inversions = calculate_inversions(start_state)
14     goal_inversions = calculate_inversions(goal_state)
15
16     # 如果初始状态和目标状态的逆序对数量的奇偶性相同，则问题可解
17     return (start_inversions % 2) == (goal_inversions % 2)
18
19 # 举例说明（返回值为True）
20 if __name__ == "__main__":
21     print(is_solvable([0,1,2,3,4,5,6,7,8],[1,2,3,4,5,6,7,8,0]))
```

五、讨论与思考

通过本次实验，我们成功地应用A*算法解决了8数码问题，并验证了该算法在处理状态空间较大、路径优化问题中的有效性。以下是我们对实验结果的讨论与思考：

1. 启发函数的选择

在实验中，我们分别尝试了曼哈顿距离、错位数、欧拉距离和切比雪夫距离作为启发函数。实验结果表明，曼哈顿距离在保证搜索效率和路径最优性方面表现最佳。这是因为曼哈顿距离能够更准确地反映每个状态到目标状态的实际移动成本，同时计算简单，不会对算法性能造成过大负担。

2. 算法性能评估

在本实验中，我们评估了DFS、BFS、Dijkstra、A*和IDA*算法在解决8数码问题中的性能。DFS虽然空间复杂度低，但无法保证找到最优解且容易陷入死胡同。BFS能够保证最优解，但空间复杂度高。Dijkstra算法通过均匀扩展找到最短路径，但效率较低。A算法结合了启发式估计和实际代价，性能最

佳，能有效减少搜索空间。IDA算法在节省内存方面表现出色，但在复杂度高的情况下可能效率较低。总体而言，A*算法在综合性能上表现最优。

3. 实际应用前景

本实验不仅展示了A算法在解决组合优化问题中的强大能力，还为其在其他实际应用中的推广提供了参考。例如，在路径规划、机器人导航、游戏AI等领域，A算法可以被广泛应用，通过合适的启发函数设计，实现高效的路径搜索和决策。

4. 未来工作展望

未来的研究工作可以进一步探索更多种类的启发函数，优化算法性能。同时，可以尝试将A*算法与其他智能搜索算法相结合，如遗传算法、模拟退火等，以提升算法在大规模复杂问题中的适应性和解决能力。

综上所述，本实验通过解决8数码问题，深入探讨了A算法的应用和性能，验证了其在智能搜索领域的有效性。我们相信，通过不断的改进和优化，A算法将在更多实际问题中展现出其独特的优势和广泛的应用前景。

六、成员分工

姓名	学号	班级	贡献
王殿云	2022211733	2022219107	8数码问题算法实现、游戏界面化设计与开发
王艺栋	2022211799	2022219108	8数码问题算法实现、文档撰写
李晓欧	2022211908	2022219108	界面化设计、文档撰写